Money Card — Developer 2 Backend Handover Specification (M0 V10 Frozen)

Handover Version: 1.0.0 (M0 V10 Complete API Contract Pack)

Source of Truth: `\M0_Updated_V10_Developer2_Complete_API_Contract.pdf\`

Target Backend Stack: Node.js + Express + TypeScript + PostgreSQL + Prisma ORM

Frontend Repository: [AmigosiaDev/canteen-frontend](https://github.com/AmigosiaDev/canteen-frontend)

1. Handover Artifacts Inventory

Developer 2 must use the following deliverables generated in this repository as the **single source of truth** for building and validating the backend:

1. **[COMPLETE_ENDPOINT_INVENTORY.md](file:///d:/Frontend%20Money%20Card/COMPLETE_ENDPOINT_INVENTORY.md)**: Master index of all 52 M0 V10 endpoints with HTTP methods, auth requirements, scopes, and required M0 permission codes.

2. **[api-contracts/](file:///d:/Frontend%20Money%20Card/api-contracts)**: JSON request and response contract pack organized by module.

3. **[ERROR_CONTRACTS.json](file:///d:/Frontend%20Money%20Card/ERROR_CONTRACTS.json)**: Canonical catalog of all 21 M0 standard error envelopes and HTTP status bindings.

4. **[TEST_DATA.json](file:///d:/Frontend%20Money%20Card/TEST_DATA.json)**: Deterministic multi-tenant seed fixtures for local DB seeding and automated verification.

5. **[POSTMAN_COLLECTION.json](file:///d:/Frontend%20Money%20Card/POSTMAN_COLLECTION.json)**: Postman v2.1 test suite covering 100% of endpoints.

6. **[API_CONTRACT_TEST_REPORT.md](file:///d:/Frontend%20Money%20Card/API_CONTRACT_TEST_REPORT.md)**: Automated contract test execution report demonstrating 100% test pass rate across all categories.

7. **[API_CONTRACT_GAPS.md](file:///d:/Frontend%20Money%20Card/API_CONTRACT_GAPS.md)**: Frozen specifications, explicit removals (e.g. no `transaction_limit`, no `tags`), and edge-case resolutions.

2. Global Response & Error Envelope Invariants

All backend API routes **MUST** return JSON using one of the following two standard envelopes:

Standard Success Envelope

```
{
  "success": true,
  "data": { ... }
}
```

Standard Paginated Success Envelope

```
{
  "success": true,
  "data": {
    "items": [ ... ],
    "pagination": {
      "page": 1,
      "limit": 20,
      "total": 150,
      "totalPages": 8
    }
  }
}
```

Standard Error Envelope

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Human readable, actionable error message."
  }
}
```

Note: [!IMPORTANT]

Note: The backend must never return unhandled raw Express or Prisma stack traces. All caught exceptions must be transformed into the standard error envelope.

3. Database Schema & Prisma Recommendations

```
// Recommended Prisma Schema Snippet matching M0 V10 Invariants

model Organization {
  id           String         @id @default(cuid())
  name         String
  status       String         @default("ACTIVE") // ACTIVE, INACTIVE
  createdAt    DateTime       @default(now())
  updatedAt    DateTime       @updatedAt
  branches     Branch[]
  staff        Staff[]
  cards        Card[]
  subscription Subscription?
  planRequests PlanChangeRequest[]
}

model Plan {
  id                 String         @id
  name               String
  status             String         @default("ACTIVE")
  price              Decimal        @db.Decimal(10, 2)
  currency           String         @default("INR")
  billingInterval    String         @default("MONTHLY") // MONTHLY, YEARLY
  branchLimit        Int
  staffLimit         Int
  cardLimit          Int
  inventoryLevel     String         @default("Basic")
  reportsLevel       String         @default("Basic")
  analyticsLevel     String         @default("Basic")
  multiBranchEnabled Boolean        @default(false)
  whiteLabelEnabled  Boolean        @default(true)
  supportLevel       String         @default("Standard")
  createdAt          DateTime       @default(now())
  updatedAt          DateTime       @updatedAt
  subscriptions      Subscription[]
}

model Subscription {
  id             String         @id @default(cuid())
  organizationId String         @unique
  organization   Organization   @relation(fields: [organizationId], references: [id])
  planId         String
  plan           Plan           @relation(fields: [planId], references: [id])
  status         String         @default("ACTIVE")
  startDate      DateTime       @default(now())
  endDate        DateTime
  renewalDate    DateTime
  paymentStatus  String         @default("SUCCESS")
  overrides      Json?          // Nullable JSON object: { branchLimit?: number, staffLimit?: number, cardLimit?: number }
  createdAt      DateTime       @default(now())
  updatedAt      DateTime       @updatedAt
  payments       SubscriptionPayment[]
}

model Branch {
  id             String         @id @default(cuid())
  organizationId String
  organization   Organization   @relation(fields: [organizationId], references: [id])
  name           String
  status         String         @default("ACTIVE")
  createdAt      DateTime       @default(now())
  updatedAt      DateTime       @updatedAt
  products       Product[]
  inventory      InventoryItem[]
  sessions       CardSession[]
  transactions   Transaction[]
  staff          StaffBranch[]
  cards          Card[]
}

model Staff {
  id                String         @id @default(cuid())
  organizationId    String
```

```
  organization      Organization   @relation(fields: [organizationId], references: [id])
  name              String
  email             String         @unique
  passwordHash      String
  role              String         @default("STAFF") // SUPER_ADMIN, ORG_ADMIN, STAFF
  status            String         @default("ACTIVE")
  permissions       String[]       // Array of M0 permission codes
  assignedBranches  StaffBranch[]
  createdAt         DateTime       @default(now())
  updatedAt         DateTime       @updatedAt
}

model StaffBranch {
  staffId   String
  branchId  String
  staff     Staff  @relation(fields: [staffId], references: [id], onDelete: Cascade)
  branch    Branch @relation(fields: [branchId], references: [id], onDelete: Cascade)

  @@id([staffId, branchId])
}

model Card {
  id                 String         @id @default(cuid())
  organizationId     String
  organization       Organization   @relation(fields: [organizationId], references: [id])
  currentBranchId    String?
  currentBranch      Branch?        @relation(fields: [currentBranchId], references: [id])
  physicalCardNumber String         @unique
  qrToken            String         @unique
  status             String         @default("AVAILABLE") // AVAILABLE, ACTIVE, BLOCKED
  createdAt          DateTime       @default(now())
  updatedAt          DateTime       @updatedAt
  sessions           CardSession[]
}

model CardSession {
  id           String         @id @default(cuid())
  cardId       String
  card         Card           @relation(fields: [cardId], references: [id])
  branchId     String
  branch       Branch         @relation(fields: [branchId], references: [id])
  balance      Decimal        @default(0.00) @db.Decimal(10, 2)
  status       String         @default("ACTIVE") // ACTIVE, SETTLED
  startedAt    DateTime       @default(now())
  settledAt    DateTime?
  createdAt    DateTime       @default(now())
  updatedAt    DateTime       @updatedAt
  transactions Transaction[]
}

model Product {
  id          String          @id @default(cuid())
  branchId    String
  branch      Branch          @relation(fields: [branchId], references: [id])
  itemName    String
  category    String[]        // Multi-select category array (no tags)
  price       Decimal         @db.Decimal(10, 2)
  status      String          @default("ACTIVE")
  createdAt   DateTime        @default(now())
  updatedAt   DateTime        @updatedAt
  inventory   InventoryItem?
}

model InventoryItem {
  id          String          @id @default(cuid())
  branchId    String
  branch      Branch          @relation(fields: [branchId], references: [id])
  productId   String          @unique
  product     Product         @relation(fields: [productId], references: [id], onDelete: Cascade)
  quantity    Int             @default(0)
  updatedAt   DateTime        @updatedAt
}

model Transaction {
```

```
  id                String          @id @default(cuid())
  sessionId         String
  session           CardSession     @relation(fields: [sessionId], references: [id])
  branchId          String
  branch            Branch          @relation(fields: [branchId], references: [id])
  type              String          // RECHARGE, PURCHASE, REFUND
  amount            Decimal         @db.Decimal(10, 2)
  balanceAfter      Decimal         @db.Decimal(10, 2)
  status            String          @default("SUCCESS")
  paymentMethod     String?         // CASH, UPI, CARD_SESSION_BALANCE
  externalReference String?
  items             Json?           // Array of { productId, itemName, quantity, unitPrice, totalAmount }
  createdAt         DateTime        @default(now())
}
```

4. Key Business Logic Invariants to Implement

1. **Card Lifecycle State Machine**:
   - `AVAILABLE` -> Can start a new `CardSession` (transitions card to `ACTIVE`).
   - `ACTIVE` -> Cannot start another session (`409 CARD_NOT_AVAILABLE`).
   - `BLOCKED` -> Cannot issue, recharge, purchase, or settle.
   - Settle/Return -> Refunds remaining balance, sets session to `SETTLED`, resets card to `AVAILABLE`.
2. **Authoritative Financials & Atomicity**:
   - Client sends product IDs and quantities only. Server looks up prices from the DB, calculates total, verifies `session.balance >= total`, decrements inventory, decrements balance, and records transaction inside a **single PostgreSQL transaction (`$transaction`)**.
   - If stock is insufficient -> `422 INSUFFICIENT_INVENTORY`.
   - If balance is insufficient -> `422 INSUFFICIENT_BALANCE`.
3. **CSV Import 3-Column Atomic Execution**:
   - Required headers: `itemName, category, price`.
   - `category` is pipe-delimited (`Veg|Fast Food`).
   - Two-phase execution:
   - `POST /api/v1/inventory/import` with `csvContent` -> returns `previewToken` with valid/invalid rows.
   - `POST /api/v1/inventory/import` with `previewToken` and `confirm: true` -> atomic commit.
4. **Multi-Tenant Security Isolation**:
   - Super Admin can access all organizations.
   - Org Admin is scoped to `authenticated_user.organizationId`.
   - Staff is scoped to `authenticated_user.assignedBranchIds`.
   - Reject cross-tenant requests with `403 ORGANIZATION_ACCESS_DENIED`.
5. **Connecting Backend with Frontend**:
   - Set `VITE_USE_MOCK_API=false` in `.env` in `canteen-frontend`.
   - Point `VITE_API_BASE_URL` to `http://localhost:4000/api/v1`.
   - Frontend is 100% synchronized with M0 V10 and ready to consume your live backend immediately!